

Table of Contents

CSE420 Compiler Design — Lab 1 Viva Cheat Sheet

Lexical Analyzer & Syntax Analyzer | Student ID: 22201344

PART 1 — THE QUESTION, IN PLAIN ENGLISH

What are we building? A mini-compiler front-end for a **subset of C**. It has two stages:

1. **Lexical Analyzer (Scanner)** — reads the source code character-by-character and groups characters into **tokens** (like INT, ID, ASSIGNOP). Think of it as breaking a sentence into words and labeling each word's part of speech.
2. **Syntax Analyzer (Parser)** — takes the stream of tokens and checks whether they form a **grammatically valid program**, according to a given grammar (like checking if a sentence's structure is valid English). It also prints out *which grammar rule matched* at every step — this is called **derivation tracing**.

Example from the spec: `int x=5;` — the lexer converts this into the token stream: `<INT> <ID,x> <ASSIGNOP> <CONST_INT,5> <SEMICOLON>`

Then the parser recognizes this token stream matches the rule: `variable_decl : type_specifier declaration_list SEMICOLON` and prints that rule plus the reconstructed code segment to the log file.

Two tools do this: - **Flex (Lex)** generates the scanner from our `.l` file. - **Bison (Yacc)** generates the parser from our `.y` file.

How they connect: Every time the parser (`yyparse()`) needs a new token, it calls `yylex()` (generated by Flex). The lexer scans characters, decides the token, and **returns** it to the parser. If the token carries a value (like an identifier's name), it's passed through a global variable `yylval`, whose type we redefined to be `symbol_info*` — a pointer to our custom class that stores a lexeme's name and type.

Required Output: A file `22201344_log.txt` containing: - Every token found, with line number and lexeme: `Line no 1: Token <INT> Lexeme int found` - Every grammar rule matched, with the line number, the rule, and the reconstructed source snippet: `At line no: 1 type_specifier : INT followed by int` - Total line count at the end.

How we solved it (walkthrough using `input.txt`):

```
int bar(int a)
{
    return a;
}
```

1. Lexer sees `int` → emits token `INT`, logs `Line no 1: Token <INT> Lexeme int found`
2. Parser matches `type_specifier : INT` → logs the rule + reconstructs `int`
3. Lexer sees `bar` → emits `ID` (with `symbol_info("bar", "ID")` attached via `yylval`)
4. Lexer sees `(, int, a, )` → tokens `LPAREN INT ID RPAREN`
5. Parser matches `param_list : type_specifier ID` → reconstructs `int a`
6. Eventually the whole function reduces up through `compound_statement`, `func_definition`, `unit`, `program`, all the way to `start`
7. At the very end, `start : program` fires, and we print `Total lines: 21`

This bottom-up reduction process (small pieces → bigger pieces) is exactly what a **bottom-up (LALR) parser** does — which is what Yacc/Bison generates.

PART 2 — THE .l FILE (LEX / FLEX FILE)

Structure of a Flex file (3 sections separated by `%%`)

Section 1: Definitions <- options, C code, regex macros

`%%`

Section 2: Rules <- pattern { action } pairs

`%%`

Section 3: User C code (optional, we don't use one here)

“`%%` means end of this part” — correct! The **first** `%%` ends the *Definitions* section and starts *Rules*. The **second** `%%` ends the *Rules* section (and would start the *User Code* section, which we leave empty).

Line-by-line explanation

%option noyywrap Tells Flex: don't require a `yywrap()` function (which normally handles multi-file input). Since we read one file only, we skip that requirement.

%{ ... } block (Definitions section, C code) Anything inside gets copied **verbatim** into the generated `lex.yy.c` file, above the scanning code. This is where we put `#includes` and global declarations the actions will need.

```
#include<bits/stdc++.h>           // all standard C++ headers
(competitive-programming style shortcut)
#include"symbol_info.h"           // our custom class to hold (name, type)
pairs
#define YYSTYPE symbol_info*      // redefine Bison's default value-stack
type to be our class pointer
#include "y.tab.h"                 // token definitions (IF, ID, INT, ...)
generated by Bison
extern YYSTYPE yylval;             // the "value" of the current token –
declared in y.tab.c, used here
using namespace std;
void yyerror(char *);             // forward declare error handler
(defined in .y file)
```

```

string loglist;           // holds each formatted log line before
writing
extern int line_num;      // line counter – DEFINED in the .y
file, just referenced here
extern ofstream outlog;   // the output log file stream – also
defined in the .y file

```

Why extern? Both `line_num` and `outlog` need to be shared between the lexer and the parser (parser also prints “At line no: X ...” using the same counter/file). To avoid two separate copies, we declare them **once** with real storage in `22201344.y`, and just reference them here with `extern`.

Regular Definitions (macros for reuse):

```

delim    [ \t\v\r]      // any single "blank" character:
space, tab, vertical tab, carriage return
ws       {delim}+        // one or more delimiters =
whitespace
newline  \n              // literally, a newline character
letter_  [A-Za-z_]       // any letter, upper or lower, or
underscore
digit    [0-9]           // any single digit
id       {letter_}({letter_}|{digit})* // identifier = starts with
letter/underscore, then any mix
integer  {digit}+        // one or more digits = an
integer literal
exponent [Ee][+-]?{digit}+ // e.g. E10, e-5, E+3 –
scientific notation part
float1   {digit}+\.{digit}*{exponent}? // e.g. 3.14159, 314. ,
3.14E-10
float2   \.{digit}+{exponent}? // e.g. .314159
float3   {digit}+{exponent} // e.g. 314159E10 (no decimal
point, but has exponent)
floatnum {float1}|{float2}|{float3} // any of the three float
forms

```

These match the spec’s rule exactly: “Numbers like *3.14159*, *3.14159E-10*, *.314159* and *314159E10* are floating point constants.”

The Rules section — pattern → action

Every rule follows the same skeleton:

```

pattern  {
        (optionally) build a symbol_info object and assign to
yyval
        build the log string
        write it to outlog
        return TOKEN_NAME;
}

```

Whitespace/newline rules (do nothing but count):

```
{ws}      { }                // whitespace: skip, no token
returned
{newline}  { line_num++; }    // newline: bump the line counter,
no token returned
```

This matches spec 3.1.5/3.1.6 — whitespace/newlines aren't tokens, but newlines must be **counted**.

Keyword rules — fixed strings that return their token, e.g.:

```
if      { loglist = "Line no "+to_string(line_num)+": Token <IF> Lexeme
"+yytext+" found\n\n";
        outlog << loglist;
        return IF; }
```

- yytext = the actual matched text (here, always "if")
- We build a string with the line number, token name, and lexeme
- Write it to the log file
- return IF; — hands the token type back to the parser (Bison auto-numbers IF from our %token declaration)

Note: Keywords like int, float, void, char ALSO attach a symbol_info object to yylval, because the parser's type_specifier rules need to read back the actual text ("int", "float", etc.) to reconstruct the source in the log. Pure keywords like if/while/return don't need this because the parser never needs their literal text back.

Identifier / constant rules — carry data via yylval:

```
{id}      {
            symbol_info *s = new symbol_info((string)yytext, "ID");
            yylval = (YYSTYPE)s;
            loglist = "Line no "+to_string(line_num)+": Token <ID>
Lexeme "+yytext+" found\n\n";
            outlog << loglist;
            return ID;
        }
```

- We new a symbol_info on the heap (must persist after this function returns — it's read later by the parser)
- Store the object's pointer into yylval (cast to YYSTYPE, which we #defined as symbol_info*)
- Return the ID token — Bison now has both the token type (ID) **and** its value (yylval, i.e. \$\$ in the matching grammar rule)

Same pattern for {integer} → CONST_INT, {floatnum} → CONST_FLOAT.

Operators (note ordering matters — longest match first, or explicit patterns):

```

"++"      { ... return INCOP; }
"--"      { ... return DECOP; }
"+"|"-"   { ... return ADDOP; }      // must come AFTER "++"/"--"
since Flex prefers longest match anyway,
                                           // but explicit separate rules
avoid ambiguity
"*"|"/"|"%" { ... return MULOP; }
"<="|">="|"=="|"!="|"<"|>" { ... return RELOP; }
"="       { ... return ASSIGNOP; }
"&&"|"||"  { ... return LOGICOP; }
"!"       { ... return NOT; }

```

Flex’s “maximal munch” rule: when multiple patterns can match, Flex always picks the **longest possible match**. So ++ will never be split into two + tokens — Flex automatically prefers the longer match even without ordering tricks. Ordering only matters as a *tie-breaker* when two patterns match the **same length** string (then the rule listed **first** wins) — e.g. if could theoretically match both the if keyword rule and the {id} rule; since the keyword rule is listed **before** {id} in the file, if correctly becomes IF, not ID.

Punctuators:

```

"(" return LPAREN;   ")" return RPAREN;
 "{" return LCURL;    "}" return RCURL;
 "[" return LTHIRD;   "]" return RTHIRD;
 ",", return COMMA;   ":" return COLON;   ";" return SEMICOLON;

```

All follow the identical logging pattern as above.

Catch-all rule:

```

. { }

```

Matches any single character not matched by anything else (Flex’s default “echo it” behavior is silenced here) — safety net so unexpected characters don’t crash the scanner.

PART 3 — THE .y FILE (YACC / BISON FILE)

Structure of a Bison file (same 3-section idea)

```

%{ C declarations %}
%token / %nonassoc declarations
%%
Grammar rules
%%
C code (main(), yyerror(), etc.)

```

Line-by-line explanation

Prologue (%{ ... %}):

```

#include "symbol_info.h"
#define YYSTYPE symbol_info*      // same redefinition as in .l – MUST
match exactly
int yyparse(void);
int yylex(void);
extern FILE *yyin;                // Flex reads from this file pointer
ofstream outlog;                  // ACTUAL definition (storage) of
the log stream
int line_num = 1;                  // ACTUAL definition of the line
counter, starts at 1
void yyerror(char *s) {           // called automatically by yyparse()
on a syntax error
    outlog << "Error at line " << line_num << ": " << s << endl;
}

```

%token declarations:

```

%token IF ELSE FOR WHILE RETURN INT FLOAT VOID CHAR DOUBLE
%token SWITCH DO BREAK DEFAULT CONTINUE CASE GOTO
%token ID CONST_INT CONST_FLOAT PRINTLN
%token LPAREN RPAREN LCURL RCURL LTHIRD RTHIRD COMMA COLON SEMICOLON
%token ADDOP MULOP INCOP DECOP RELOP ASSIGNOP LOGICOP NOT

```

Declares every terminal symbol (token) the parser expects from the lexer. Bison auto-assigns each one an internal integer ID and writes matching #defines into `y.tab.h` — this is exactly what `#include "y.tab.h"` in the `.l` file picks up, so both files agree on token numbers.

%nonassoc — solving the dangling-else problem:

```

%nonassoc LOWER_THAN_ELSE
%nonassoc ELSE

```

This is the classic **if-else ambiguity**: in

```
if (a) if (b) s1; else s2;
```

does the `else` belong to the inner `if` or the outer `if`? Standard C (and our spec) says it always binds to the **nearest unmatched if**. Declaring `ELSE` with **higher precedence** than a dummy token `LOWER_THAN_ELSE` (attached via `%prec` on the plain-if rule) tells Bison: “*when in doubt, shift (i.e., prefer to consume the else) rather than reduce the plain if early.*” This resolves the shift/reduce conflict cleanly, giving us **0 conflicts** on yacc --verbose.

Grammar rules section

Every rule follows:

```

lhs : rhs_symbols
{
    outlog << "At line no: " << line_num << " lhs : rhs_symbols "
<< endl << endl;
    outlog << <reconstructed code string> << endl << endl;
    $$ = new symbol_info(<reconstructed code string>, "lhs_type");
}

```

```
}
;
```

Key notation: - \$\$ = the value (symbol_info) of the **left-hand side** being built right now — this becomes \$1, \$2, etc. when this rule is later used inside a larger* rule. - \$1, \$2, \$3... = the values of the **right-hand side symbols**, in left-to-right order, whether they're tokens (from yylval) or other grammar rules (from their own \$\$). - >getnameofsymbol() = pulls the actual lexeme text (like "int", "x", "56") out of a symbol_info object.

Example — type_specifier:

```
type_specifier : INT
{
    outlog<<"At line no: "<<line_num<<" type_specifier : INT
"<<endl<<endl;
    outlog<<"int"<<endl<<endl;
    $$ = new symbol_info("int","type_specifier");
}
```

Here \$1 (the INT token's yylval) isn't even used — we just hard-code "int" since we know exactly what text an INT token represents. \$\$ is a brand new symbol_info labeled with type "type_specifier" so that a parent rule (like func_definition) can later call \$1->getnameofsymbol() to get back "int".

Example — string concatenation rules (this is the “reconstruction” logic):

```
declaration_list : declaration_list COMMA ID
{
    outlog<<"At line no: "<<line_num<<" declaration_list :
declaration_list COMMA ID "<<endl<<endl;
    outlog<<$1->getnameofsymbol()+" , "+$3-
>getnameofsymbol()<<endl<<endl;
    $$ = new symbol_info($1->getnameofsymbol()+" , "+$3-
>getnameofsymbol(),"declaration_list");
}
```

- \$1 = the existing declaration_list's built-up string so far (e.g. "x, y")
- \$2 is the COMMA token — we don't need its value, we just insert a literal " , "
- \$3 = the newly matched ID's name (e.g. "z")
- Result: "x, y, z" — exactly reconstructing the original source text, which then gets logged and passed further up

This exact pattern (concatenate children's text with the right literal punctuation/spacing in between) repeats through **every single rule** in the grammar — param_list, compound_statement, statement, expression, factor, etc. Once you understand this one example, you understand the whole file's logic.

Example — recursive statements (building up a block line by line):

```
statements : statement { $$ = new
symbol_info($1->getnameofsymbol(),"statements"); }
```

```

        | statements statement          { $$ = new
symbol_info($1->getnameofsymbol()+"\n"+$2-
>getnameofsymbol(),"statements"); }
;

```

First statement in a block — base case. Every subsequent statement — append with a newline, mimicking multi-line source code exactly as it appeared.

Example — the dangling-else fix in action:

```

statement : IF LPAREN expression RPAREN statement %prec
LOWER_THAN_ELSE
        { ... "if("+ $3->getnameofsymbol()+" )\n"+$5-
>getnameofsymbol() ... }
        | IF LPAREN expression RPAREN statement ELSE statement
        { ... "if(...)\n"+$5+"\nelse\n"+$7 ... }

```

The %prec LOWER_THAN_ELSE on the plain-if rule is what tells Bison to prefer shifting an incoming ELSE (attaching it to the closest if) instead of reducing the shorter rule prematurely.

Function calls — argument_list can be empty:

```

argument_list : arguments { $$ = new symbol_info($1-
>getnameofsymbol(),"argument_list"); }
        | { $$ = new
symbol_info("", "argument_list"); } // empty production!
;

```

The second alternative has **nothing** on the right-hand side — this matches a function call with zero arguments, like foo(). \$\$ becomes an empty string so ID LPAREN argument_list RPAREN reconstructs cleanly as foo() and not foo(null).

main() — the driver

```

int main(int c, char *v[])
{
    if(c != 2) { cout<<"Please provide input file"<<endl; return 0; }
    yyin = fopen(v[1], "r"); // open the file named
on the command line
    outlog.open("22201344_log.txt", ios::trunc); // create/overwrite
the log file
    if(yyin == NULL) { cout<<"Couldn't open file"<<endl; return 0; }
    yyparse(); // starts the whole
lex+parse process
    outlog.close();
    fclose(yyin);
    return 0;
}

```

- argc must be 2: program name + input filename (e.g. ./a.out input.txt)

- Where does “Total lines: N” get printed? Inside the top-level start : program action:**

This fires exactly once, after the entire file has been successfully parsed — `line_num` at that point equals the total number of newlines encountered (i.e., total lines).

symbol_info.h

This is the **glue** between lexer and parser — every `yyval` and every `$$` is a pointer to one of these objects. It's intentionally simple: just a name + a type, with getters.

The build pipeline, run top-to-bottom:

```
yacc -d -y --debug --verbose 22201344.y # generates y.tab.c AND
y.tab.h (the -d flag)
g++ -w -c -o y.o y.tab.c # compile parser to object
file
flex 22201344.l # generates lex.yy.c (needs
y.tab.h to know token names)
g++ -fpermissive -w -c -o l.o lex.yy.c # compile scanner to object
file
g++ y.o l.o # link both object files
into a.out
```

```
./a.out input.txt # run: parse input.txt,
produce 22201344_log.txt
```

Why this order? Yacc must run first because it generates `y.tab.h`, which the `.l` file `#includes` (for the token `#defines`). Flex can only compile successfully once that header exists.

input.txt

Sample C-subset source program used to test the whole pipeline — contains a function `bar`, a `main` with an if-else-if-else chain, demonstrating declarations, assignment, relational/modulo operators, and the `printf` pseudo-statement.

PART 5 — OUTPUT WALKTHROUGH (using `input.txt`, line by line)

Recall the input:

```
1 int bar(int a)
2 {
3     return a;
4 }
5 (blank)
6 int main()
7 {
8     a = 56;
9     if(a%2 == 0)
10    {
11        printf(a);
12    }
13    else if (a%2 == 1)
14    {
15        printf(a);
16    }
17    else
18    {
19        printf(a);
20    }
21 }
```

Line 1 — `int bar(int a)` 1. Scanner: `int` — `<INT>` token — parser immediately reduces `type_specifier : INT` — prints `int` 2. Scanner: `bar` — `<ID>` token (just logged, not reduced yet — waiting for more context) 3. Scanner: `(` — `<LPAREN>` 4. Scanner: `int` — `<INT>` — reduces `type_specifier : INT` again (this one is for the parameter) 5. Scanner: `a` — `<ID>` — parser reduces `param_list : type_specifier ID` — prints `int a` 6. Scanner: `)` — `<RPAREN>`

Line 2 — `{` Scanner emits `<LCURL>`. Parser doesn't reduce anything yet — waiting for the body.

Line 3 — return a; 1. return → <RETURN> 2. a → <ID> → reduces bottom-up through the **entire expression chain**: variable : ID → factor : variable → factor_info : factor → unary_expression : factor → term : unary_expression → simple_expression : term → rel_expression : simple_expression → logic_expression : rel_expression → expression : logic_expression. Each of these is a real reduction because Yacc's grammar defines expressions through this whole hierarchy (this enforces operator precedence — arithmetic binds tighter than relational, which binds tighter than logical). 3. ; → <SEMICOLON> → reduces statement : RETURN expression SEMICOLON → prints return a; 4. Immediately also reduces statements : statement → prints return a;

Line 4 — } <RCURL> → reduces compound_statement : LCURL statements RCURL → prints the whole block:

```
{
return a;
}
```

Then immediately reduces func_definition : type_specifier ID LPAREN param_list RPAREN compound_statement → prints the whole function, then unit : func_definition, then program : unit (first function becomes the base of program).

Line 6 — int main() Same pattern as bar, but with **no parameters**, so the alternate rule fires: func_definition : type_specifier ID LPAREN RPAREN compound_statement (no param_list).

Line 8 — a = 56; 1. a = → scanner emits <ID> then <ASSIGNOP>; parser reduces variable : ID 2. 56 → <CONST_INT> → reduces factor : CONST_INT → up through the same expression chain as before, ending at logic_expression : rel_expression 3. ; → reduces expression : variable ASSIGNOP logic_expression → prints a=56, then expression_statement : expression SEMICOLON → a=56;, then statement : expression_statement, then statements : statement (first statement in this block).

Line 9 — if(a%2 == 0) 1. a%2 → builds up term : term MULOP unary_expression → prints a%2 2. == 0 → the RELOP combines two simple_expressions into rel_expression : simple_expression RELOP simple_expression → prints a%2==0 3. This becomes logic_expression, then expression — but the IF () and the statement after it haven't reduced yet; parser is still waiting to see the full if(...) body.

Lines 10-12 — { printf(a); } 1. printf (a) ; → reduces directly to statement : PRINTLN LPAREN ID RPAREN SEMICOLON → prints printf(a); 2. statements : statement 3. } closes it → compound_statement : LCURL statements RCURL → prints the block, then statement : compound_statement

Line 13 — else if (a%2 == 1) Because of the dangling-else %prec fix, this else correctly attaches to the **nearest** if — the parser doesn't finish the outer if statement yet; it keeps shifting, parsing this as a **nested if** inside the else-branch.

Lines 14-16 — same printf(a); block pattern as before, for the a%2==1 case.

Line 17-20 — else { printf(a); } This closes the **inner** if: statement : IF LPAREN expression RPAREN statement ELSE statement fires for the inner if(a %2==1) ...else..., producing the reconstructed nested if-else text. Then, because that whole nested if-else **is itself** the statement following the outer else, the **outer** if-else rule fires next: statement : IF LPAREN expression RPAREN statement ELSE statement fires again — this time for if(a%2==0) ... else if(a%2==1)...else..., chaining correctly.

Then statements : statements statement appends this whole if-else-if-else chain onto a=56;.

Line 21 — final } Closes main's body — compound_statement, then func_definition : type_specifier ID LPAREN RPAREN compound_statement, then unit : func_definition, then program : program unit (now combining bar's unit with main's unit — this is the **second** function, so the recursive rule program : program unit fires, not the base case).

Very end — start : program Fires once, after all input is consumed. Prints:

At line no: 21 start : program

Total lines: 21

QUICK-FIRE VIVA Q&A

Q: Why do we need #define YYSTYPE symbol_info* in BOTH files? A: YYSTYPE is the type of the value each token/rule carries (yyval and \$\$/\$1/\$2...). Both Flex and Bison compile *separately* into lex.yy.c and y.tab.c, but they must **agree** on this type — otherwise yyval would be interpreted differently by each file, causing garbage data or a compile error.

Q: What is yytext? A: A global char array (auto-managed by Flex) holding the **exact substring** that matched the current pattern — e.g. if the rule {id} matches foobar, yytext = "foobar".

Q: What's the difference between a token and a lexeme? A: Lexeme = the actual text ("if", "56", "x"). Token = the category/class it belongs to (IF, CONST_INT, ID).

Q: Why does the parser print rules “bottom-up”? A: Because Bison generates an **LALR(1) bottom-up shift-reduce parser** — it builds the parse tree from the leaves (individual tokens) upward to the root (start), reducing small pieces into bigger grammar symbols as it goes. This is why type_specifier : INT prints *before* func_definition : ... — the smaller piece must exist before it can be used inside the larger rule.

Q: What does -d do in yacc -d -y ...? A: Tells Bison to also generate the y.tab.h header file (containing #define token constants), which the .l file needs.

Q: How is operator precedence enforced without %left/%right declarations? A:

Through the **grammar's own hierarchy**: `expression` → `logic_expression` → `rel_expression` → `simple_expression` → `term` → `unary_expression` → `factor`. Each level can only combine using operators appropriate to that level (`||/&&` only at `logic_expression`, `+/-` only at `simple_expression`, `*///%` only at `term`), so precedence is baked into the grammar structure itself rather than needing precedence declarations.

Q: How was the dangling-else ambiguity resolved? A: `%nonassoc LOWER_THAN_ELSE / %nonassoc ELSE`, with `%prec LOWER_THAN_ELSE` attached to the plain `if` rule — this tells Bison to prefer shifting the `ELSE` token (binding it to the nearest `if`) rather than reducing early, eliminating the shift-reduce conflict.

Q: How many conflicts does your grammar have? A: Zero — verified via `yacc -d -y --verbose` and inspecting `y.output`.

Q: Where does line counting happen — lexer or parser? A: The **counter variable** (`line_num`) is *defined* in the `.y` file (so both files can share it via `extern`), but it's *incremented* inside the `.l` file's `{newline}` rule, since only the lexer sees raw characters (newlines).